



The Iby and Aladar Fleischman Faculty of Engineering
Electronics - Laboratory B

FPGA Experiment Report - Stopwatch

Student 1:

Name: Mahmoud Stitia

ID: 214032682

Student 2:

Name: Jihad Abu Daya

ID: 318791068

Table of Contents

1	Full Adder (FA)	2
1.1	Truth Table	2
1.2	Gate-Level Schematic	2
1.3	Waveform Simulations	3
2	Conditional Sum Adder (CSA(N))	4
2.1	Block Diagram	4
2.2	Time Comparison with Ripple Carry Adder	5
2.3	Generate Block	5
2.4	Simulation Waveform - CSA(3)	5
2.5	Simulation Waveform - CSA(4)	7
3	Limited Incrementor(Lim_Inc)	8
3.1	Block Diagram	8
4	Debouncer	10
4.1	Block Diagram	10
4.2	Counter Value Tradeoff	11
5	Seven Segment Display (Seg_7_Display)	12
5.1	Clock Division Selection	12
6	Stopwatch	13
6.1	Block Diagram	13
6.2	Elaborated Design Schematic	14
6.3	Synthesized Design Comparison	15
7	Timing and summary	16
7.1	Timing Summary	16
7.2	Critical Setup Path	17
7.3	Project Summary	18

1 Full Adder (FA)

1.1 Truth Table

Question: Write the truth table for the Full Adder, showing inputs a, b, ci and outputs sum, co. Which rows produce a carry out?

a	b	ci	sum	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Full Adder Truth Table

Rows that produce carry out: Rows with at least two 1's in the input will indeed produce a high carry out, as we can tell from the table when looking at lines number 4,6,7,8.

1.2 Gate-Level Schematic

Question: Draw a schematic showing only AND, OR, XOR gates that implement your FA.

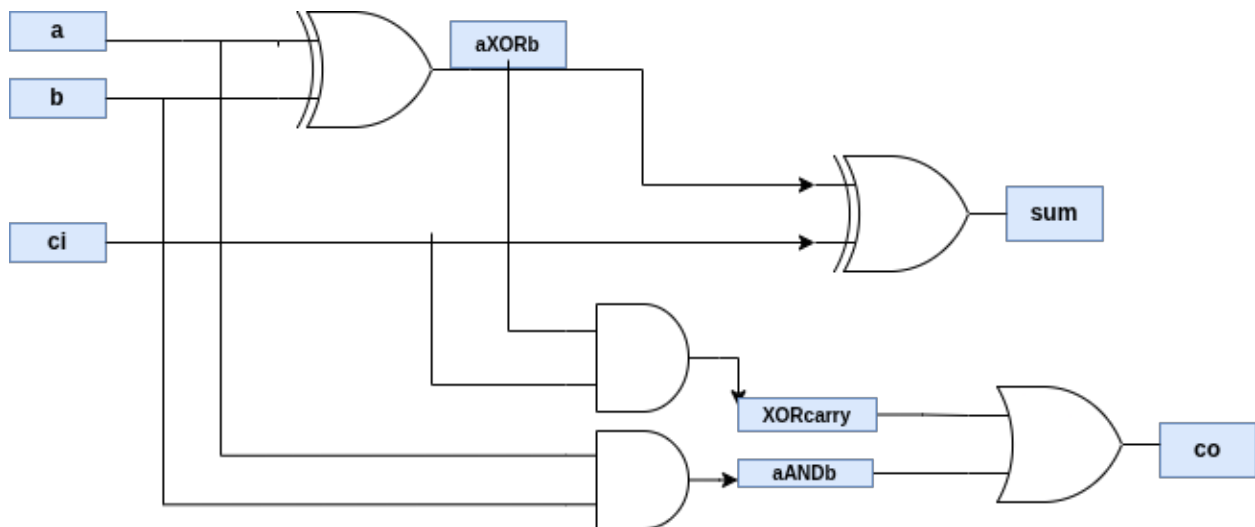


Figure 1: Full Adder Gate-Level Schematic

1.3 Waveform Simulations

Question: Provide an annotated waveform of your simulation.

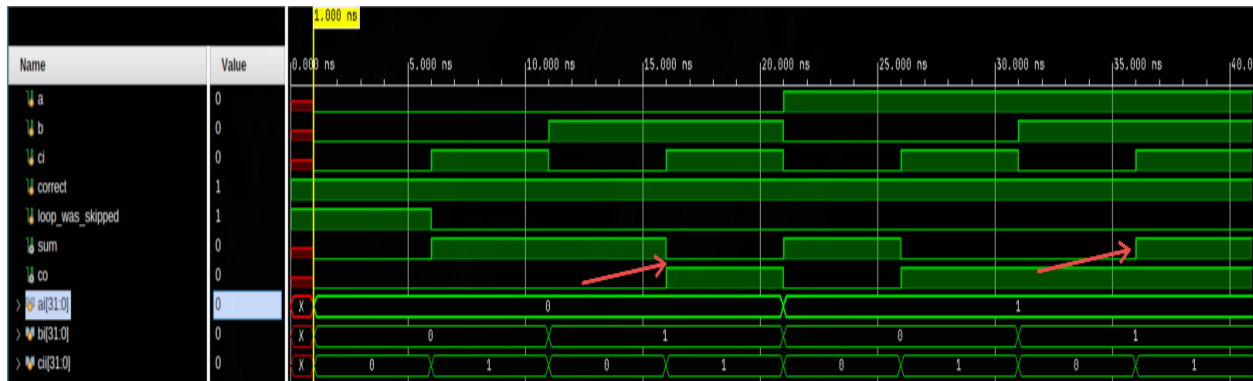


Figure 2: Detailed view of the Full Adder simulation.

Explanation of the waveform: First of all, as we can see, correct stayed at '1' for the whole duration of the simulation indicating that the FA is working as intended. The first line represents the value of “a”. The second line represents the value of “b”. The third line represents the value of “ci”. The fourth and fifth lines represent the values of “correct” and “loop_was_skipped”.

We use these two values to verify whether our calculation is correct. The bottom two lines represent the value of “sum” and the value of “co” (carry-out) respectively; these values are the final outputs from our design where:

- **i:** The first red arrow points to the transition at 15 ns. At this moment, the input ci switches from 0 to 1, while a and b remain at 0. This results in a total sum of 1. Consequently, the sum output changes from 0 to 1 to reflect this addition.
- **ii:** The second red arrow illustrates the transition at 35 ns. At this point, the input a is 1, b is 0, and ci switches from 0 back to 1. Because the sum equals 2 (represented as binary 10), the sum bit drops back to 0. Simultaneously, the arrow indicates that co (carry out) transitions from 0 to 1 to represent the carry bit.

2 Conditional Sum Adder (CSA(N))

2.1 Block Diagram

Question: Design a block diagram for the module, use instantiated modules, show and explain any other logic necessary.

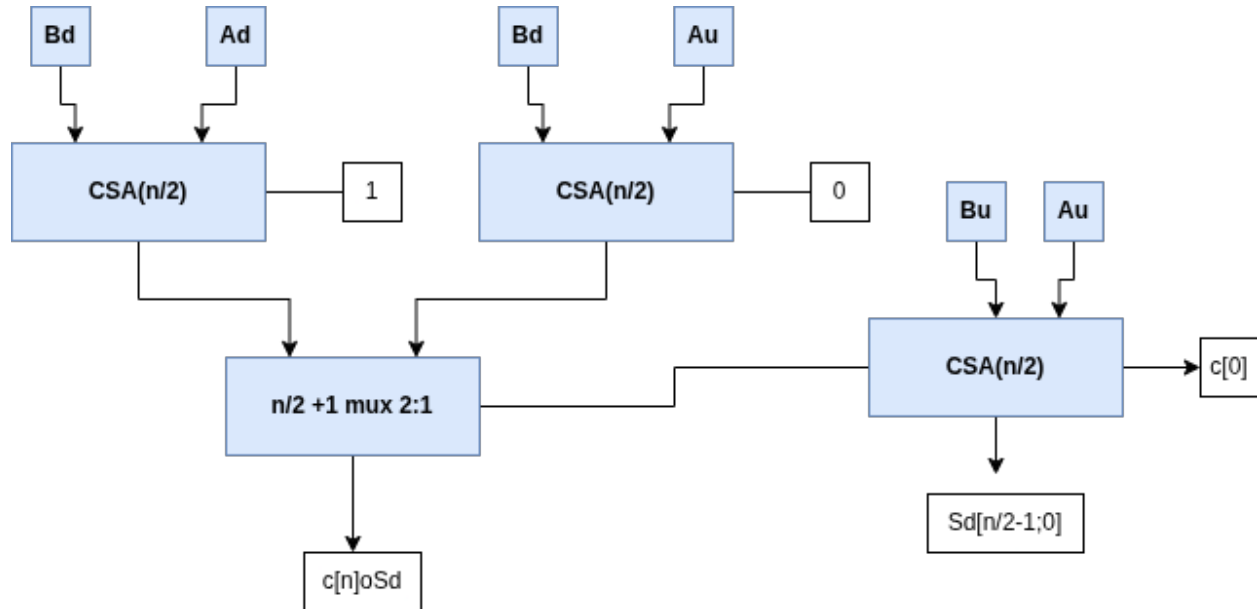


Figure 3: CSA Block Diagram

Block Diagram Explanation: This is the classic CSA block diagram that we have seen in the course "Digital Logic Systems".

It adds parallelism to the addition operation, which therefore makes the addition operation much faster, but this great advantage of speed comes at the cost of space. How it works and where is the quick part? When adding two numbers each of length of more than a single bit, in order to compute the result for each two added bits, one must include in the addition the carry bit that generated from adding the two previous bits, the problem emerges from the fact that the carry bit is not ready yet. For the desired carry bit to be ready, the two current bits should "wait for it" to be calculated after a series of sequential addition that should have taken place previously. which results in slowing down the addition speed (like the Ripple Carry Adder for example), so alternatively, the CSA suggests to calculate all the addition operations in advance twice, once assuming $c_i = 0$ and once assuming $c_i = 1$. At the end, the correct result is chosen using a multiplexer with the select bit being the c_o output-bit of the CSA that just finished adding the two right-hand side parts of the two bit string inputs. This design is recursive, with $CSA(1)$ being the FA module we designed earlier.

2.2 Time Comparison with Ripple Carry Adder

Question: How does the Conditional Sum Adder improve timing compared to a simple ripple-carry adder? In what situations would a ripple-carry adder still be preferable?

Answer the CSA improves timing by eliminating the full N -bit carry ripple. By precomputing MSB-side sums for both possible carry-in values $c_i = 0$ and $c_i = 1$, the circuit only awaits the lower-block carry to select the result via multiplexers. Consequently, the worst-case delay is reduced to the lower-block carry generation plus the MUX chain delay, rather than a full N -stage propagation.

A ripple-carry adder is still preferable when area, power, and simplicity are more important than speed. For small bit-widths the ripple delay is probably acceptable, so the extra duplicated logic and multiplexers of a CSA are not worth the cost. RCAs are also attractive in low-power/low-area designs and in non-critical timing paths, since they have minimal hardware, lower switching activity, and are easier to implement and verify.

2.3 Generate Block

Question: What is a "generate block"? Why do we need to use it in the CSA implementation?

Answer A generate block in Verilog is an elaboration-time construct (using `generate/endgenerate` with `for/if/case`) that lets the compiler replicate or conditionally include hardware based on parameters or constants. We need it in the CSA because the adder is built from repeated bit-slices/blocks. Instead of manually instantiating the same logic N times, a generate loop creates these repeated instances parameterically, making the design scalable. to any bit-width and consistent across all stages

2.4 Simulation Waveform - CSA(3)

Question: Provide an annotated waveform of your simulation. How can you confirm from the waveform that the "conditional" part of the design works correctly?

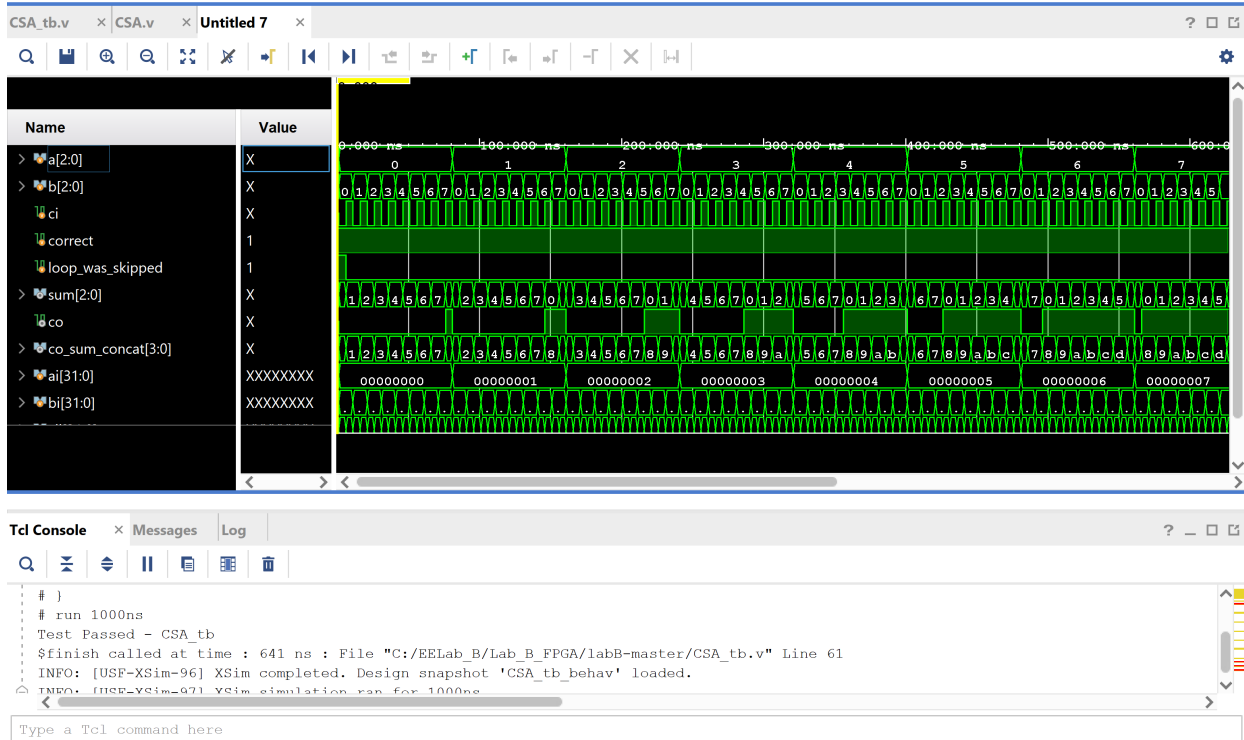


Figure 4: CSA(3) Simulation Waveform - Full Simulation

The simulation of the CSA(3) indeed succeeded and we got a "Test Passed - CSA-tb" message in the TCL console, also another point that worth mentioning here as it is visible from the waveform that the correct signal remained in "high" throughout the entire simulation time which is another indicator of success.

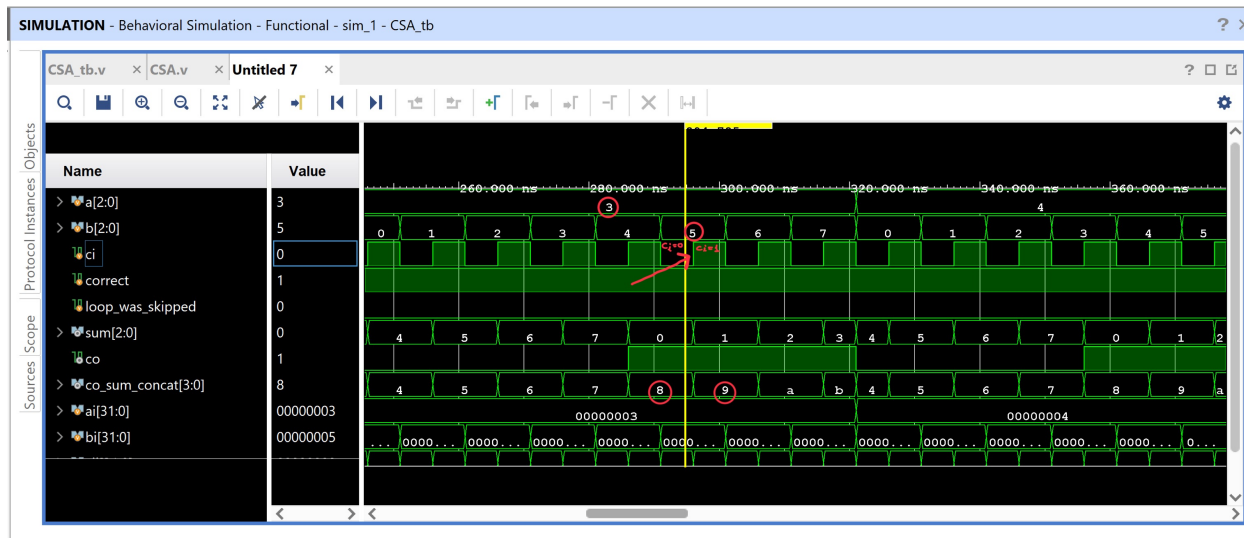


Figure 5: CSA(3) Simulation Waveform (Annotated)

2.5 Simulation Waveform - CSA(4)

Referring to the previous section Revisiting the previous section but this time with the parameter N set to 4 in the CSA implementation, and the parameter Width set to 4 in the CSA-testbench.

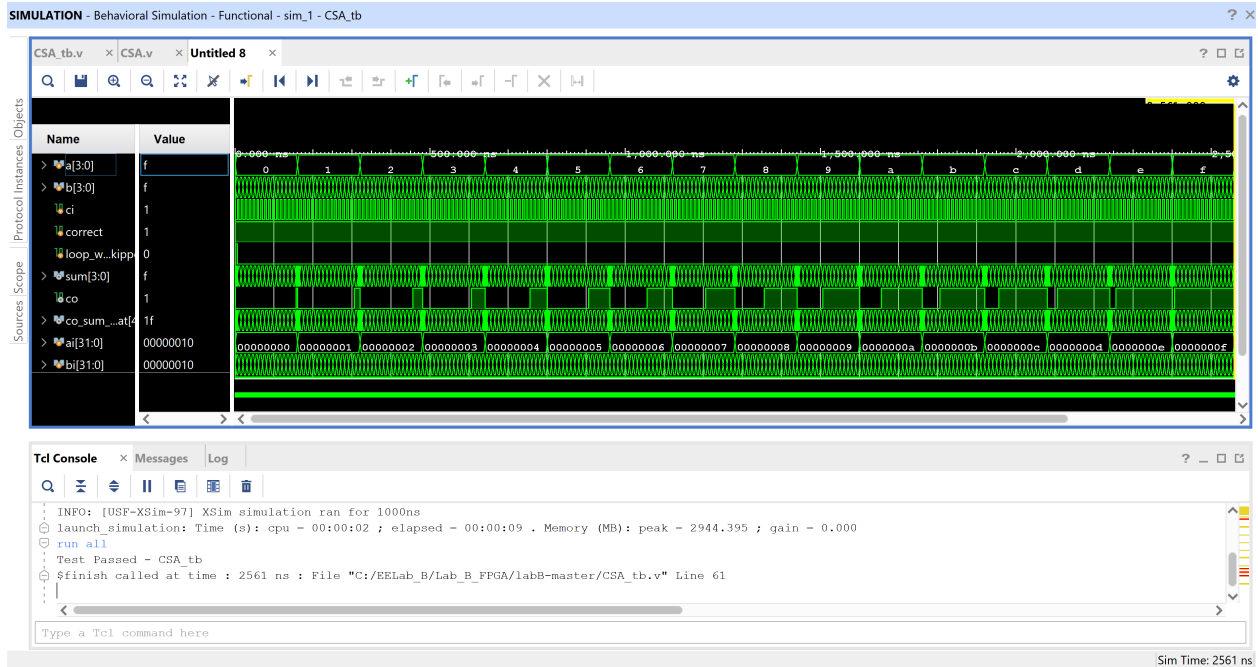


Figure 6: CSA(4) Simulation Waveform - Full Simulation

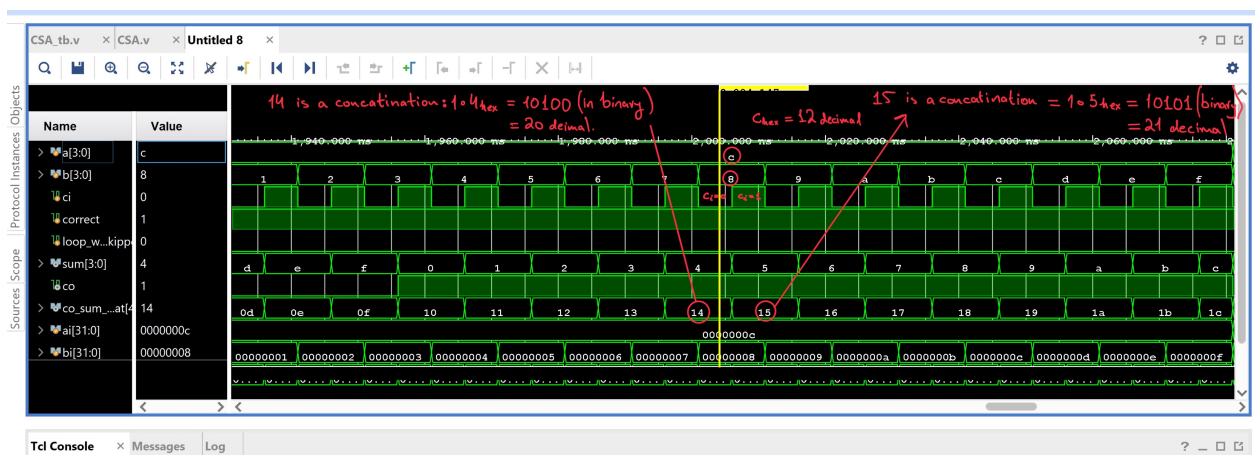


Figure 7: CSA(4) Simulation Waveform (Annotated)

3 Limited Incrementor(Lim_Inc)

3.1 Block Diagram

Question: Design a block diagram for the module, use instantiated modules, show and explain any other logic necessary.

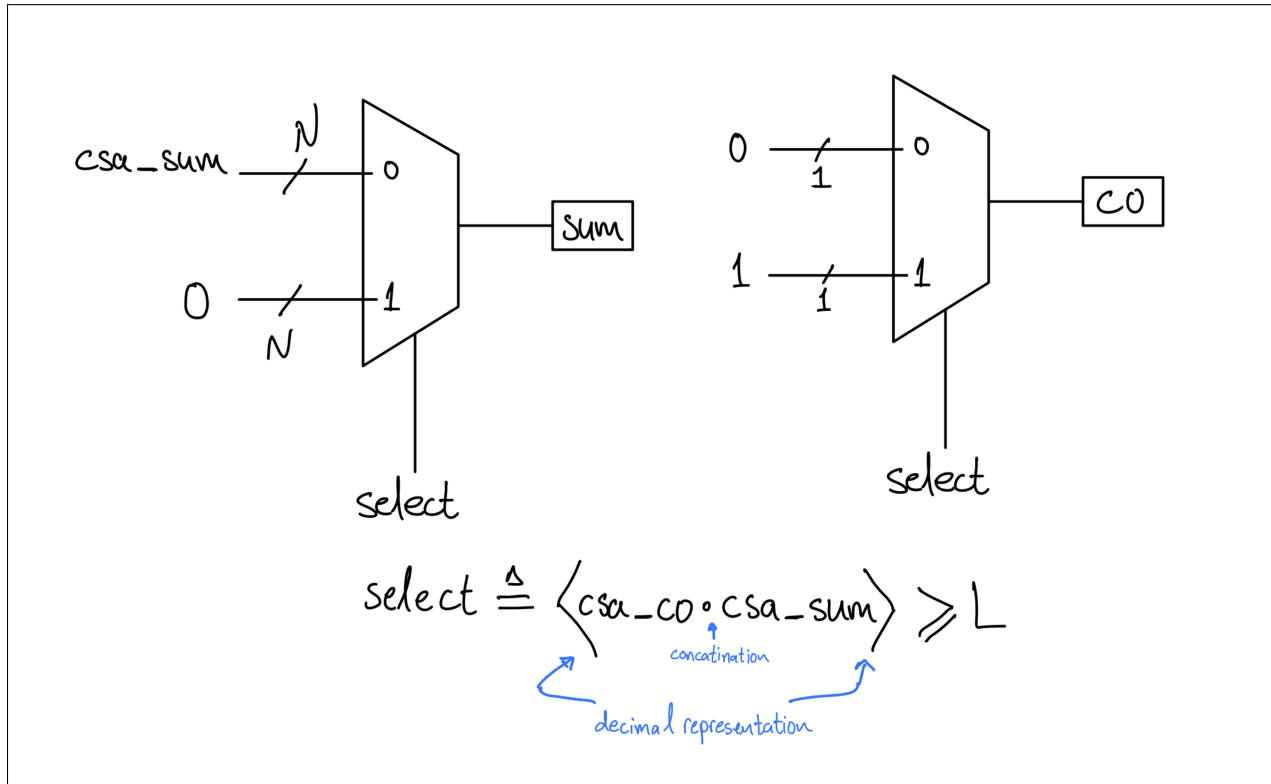


Figure 8: Limited Incrementor Block Diagram

Block Diagram Explanation: The block diagram shows a pure conversion of the given functionality of the limited incrementer to logic. We begin by setting the selection channel to be a check if the following condition holds $\langle \text{csa_co} \circ \text{csa_sum} \rangle \geq L$, if it does (then selection = 1) then the circuit must output an N-bit length binary array of zeros as the sum, and a carry out bit that is equal to 1, which is the output of the two MUXes when they are outputting their "1" route.

Similarly, if the condition in the selection channel does not hold (i.e. select = 0), then the circuit must return the exact same input as the sum, and the carry out to be equal to 0, which is the output of the two MUXes when they are outputting their "0" route.

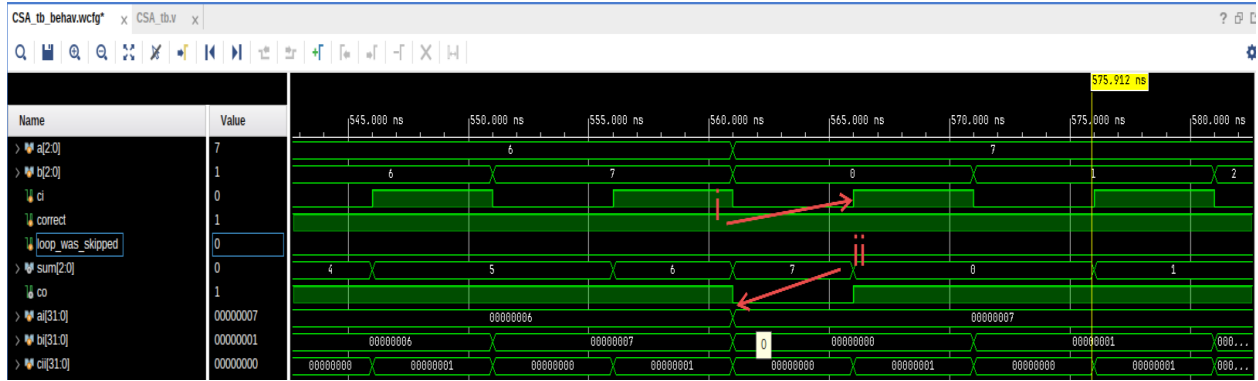


Figure 9: Zoomed in view of the simulation focusing on carry propagation.

In the waveform viewer we can see the value of each

4 Debouncer

4.1 Block Diagram

Question: Design a block diagram for the module, use instantiated modules, show and explain any other logic necessary.

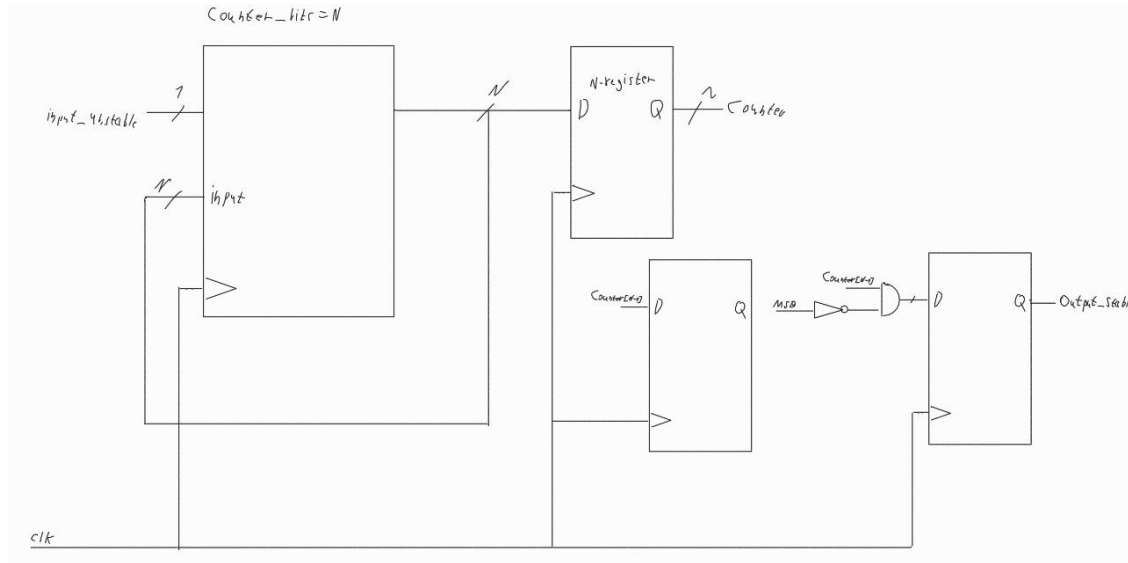


Figure 10: Debouncer Block Diagram

Block Diagram Explanation: Our debouncer uses a saturated N -bit counter register, a single flip-flop, and an AND gate with one inverted input.

On every rising clock edge, if

$\text{input_unstable} = 1$ the counter increments, and if $\text{input_unstable} = 0$ the counter decrements (saturating at 0). The flip-flop msb samples $\text{counter}[N-1]$ on each clock edge, delaying the MSB by one cycle. The output is then:

$$\text{output_stable}(t) = \overline{\text{counter}[N-1](t-2)} \& \text{counter}[N-1](t-1)$$

This implements a rising-edge detector on the counter's MSB: output_stable goes high for exactly one clock cycle at the moment the MSB transitions from 0 to 1. This ensures a clean, single-cycle pulse regardless of how long the physical button is held.

4.2 Counter Value Tradeoff

Question: Discuss the tradeoff between choosing high and low values for the counter, and how is this related to the low-pass-filter debouncing approach we saw in the previous experiments.

Answer: The `COUNTER_BITS` parameter determines the duration of the debounce interval. Increasing its value enlarges the counter, which improves immunity to noise and contact bouncing but also introduces longer latency before the system responds to a button press. Conversely, reducing this value shortens response time but increases the likelihood of false triggering caused by unstable or noisy input signals.

This tradeoff is directly analogous to the low-pass-filter-based debouncing approach from earlier experiments. A lower cutoff frequency averages the input over a longer time, suppressing bouncing at the cost of increased delay, while a higher cutoff frequency allows faster response with reduced noise suppression. Similarly, a larger counter value requires longer input stability, whereas a smaller counter value reacts faster but is more sensitive to noise.

5 Seven Segment Display (Seg_7_Display)

5.1 Clock Division Selection

Question: Describe your choice of bits for clock division and justify your answer analytically.

Answer: The provided clock runs at 100 MHz, so it ticks every 10 ns.

The `clkdiv` register increments every clock cycle, so bit n toggles every 2^n cycles, i.e. every 10×2^n ns.

Since our design drives 4 digits, we need two bits to cycle through them. The total refresh time for all 4 digits must be between 1 ms and 16 ms to avoid visible blinking. If we select bits [19:18], bit 18 changes every:

$$10 \times 2^{18} \text{ ns} = 2,621,440 \text{ ns} \approx 2.62 \text{ ms}$$

The total refresh cycle for all four digits is then $4 \times 2.62 = 10.49$ ms, which falls well within the required range of 1–16 ms.

6 Stopwatch

6.1 Block Diagram

Question: Design a block diagram for the module, use instantiated modules, show and explain any other logic necessary.

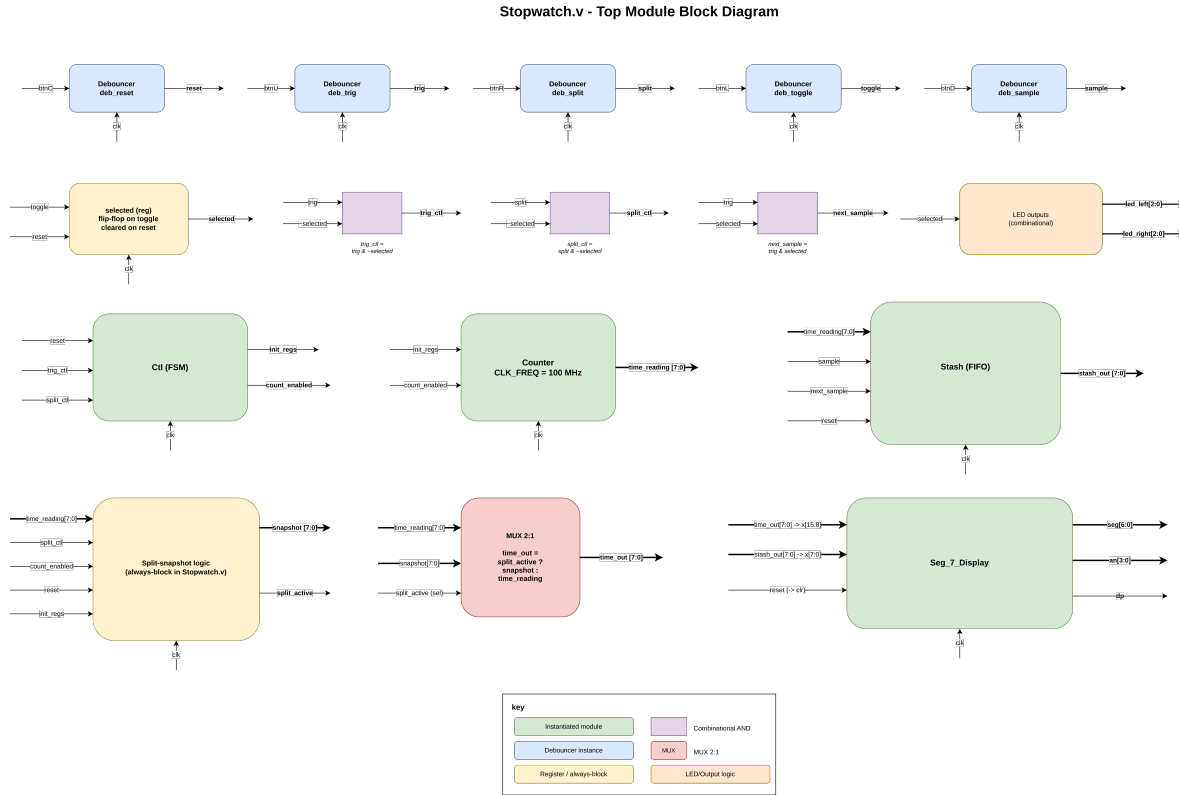


Figure 11: Stopwatch Top-Level Block Diagram

6.2 Elaborated Design Schematic

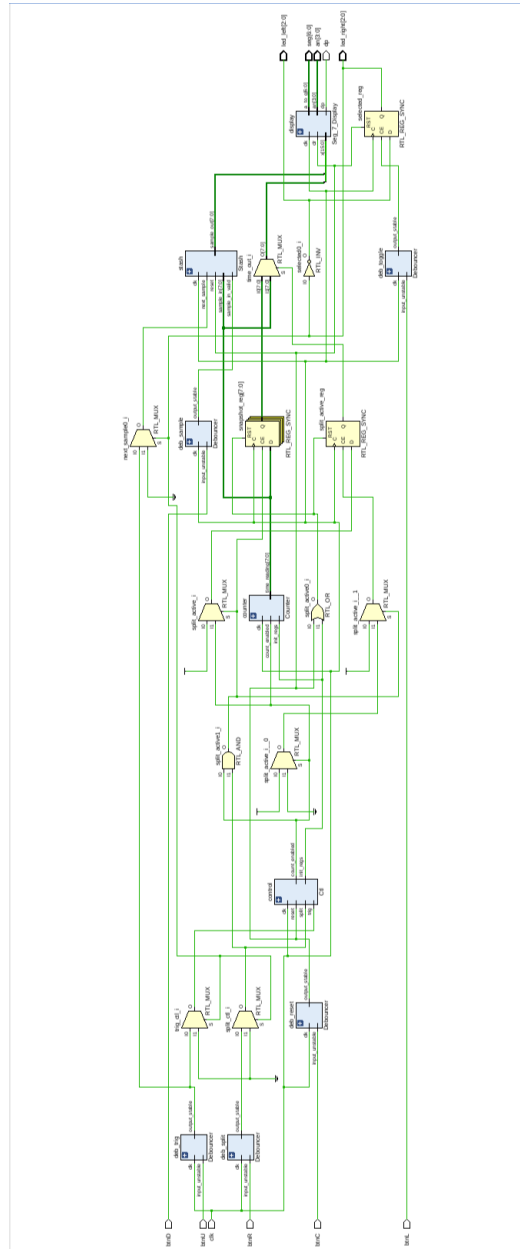


Figure 12: Elaborated Design Schematic

The elaborated design schematic shows all instantiated modules (5 Debouncers, Ctl, Counter, Stash, Seg_7_Display) with their correct interconnections. All top-level inputs (**btnC**, **btnU**, **btnR**, **btnL**, **btnD**, **clk**) and outputs (**seg**, **an**, **dp**, **led_left**, **led_right**) are present. The split logic and choose logic appear as MUXes and registers between the main modules.

6.3 Synthesized Design Comparison

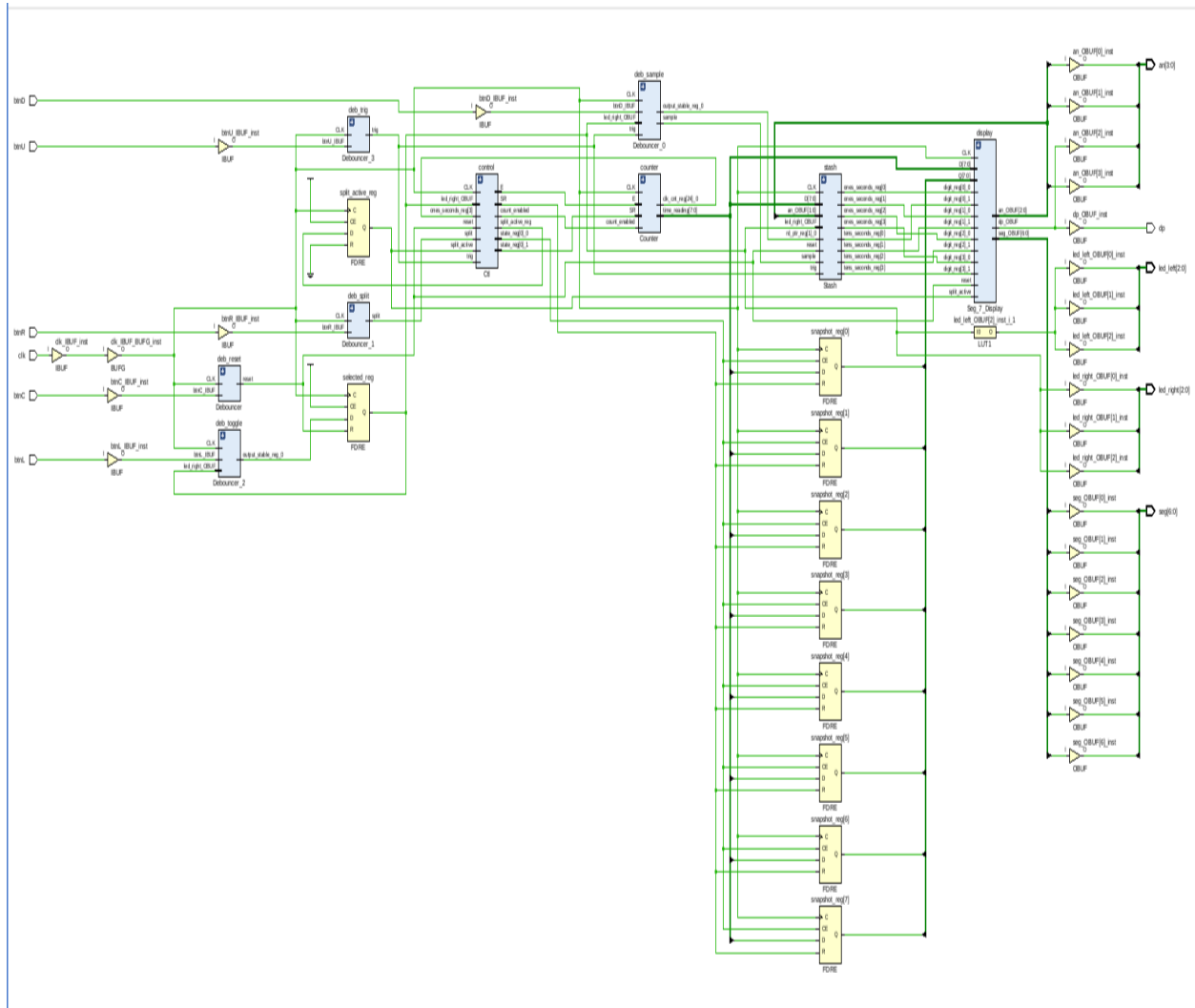


Figure 13: Synthesized Design Schematic

Differences: In the elaborated design, we see the logical/behavioral structure as written in Verilog—high-level modules, generic RTL primitives (RTL_MUX, RTL_REG, RTL_AND), and abstract signal buses. After synthesis, Vivado maps the design onto actual FPGA primitives: LUTs (Look-Up Tables), FDREs/FDSEs (flip-flops with reset/set and enable), IBUFs/OBUFs (I/O buffers), and a BUFG (global clock buffer). The synthesizer also performs optimizations such as merging logic, removing unused paths, and flattening hierarchies where beneficial.

7 Timing and summary

7.1 Timing Summary

Question: Provide a screenshot of the design timing summary exhibiting positive slack for both setup and hold times.

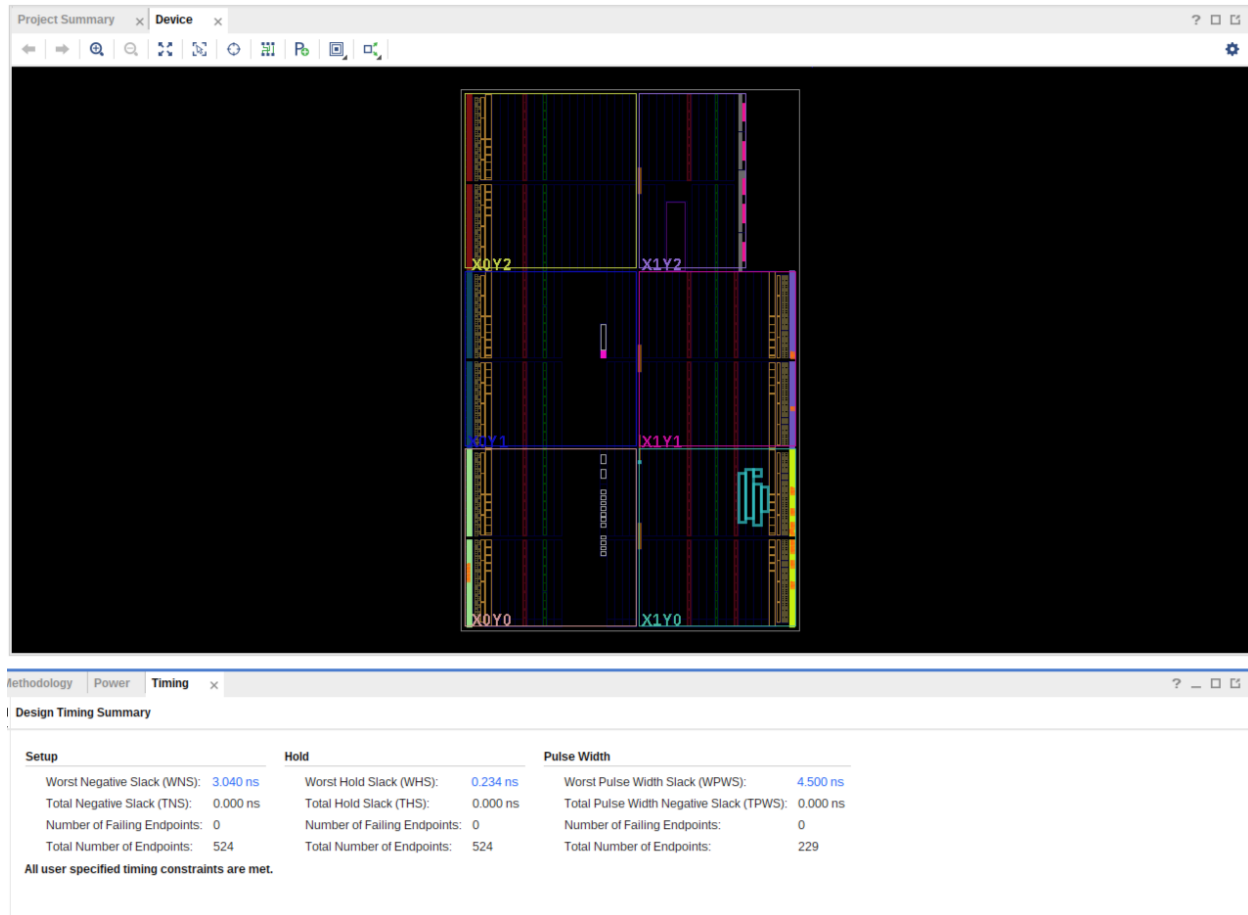


Figure 14: Design Timing Summary — all timing constraints are met.

As shown in the timing summary, all specified timing constraints are met:

- **Setup:** Worst Negative Slack (WNS) = 3.040 ns (positive $\Rightarrow$ constraint met)
- **Hold:** Worst Hold Slack (WHS) = 0.234 ns (positive $\Rightarrow$ constraint met)
- **Pulse Width:** Worst Pulse Width Slack (WPWS) = 4.500 ns (positive $\Rightarrow$ constraint met)

7.2 Critical Setup Path

setup path.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	3.040	5	6	29	counterclk_cnt_reg[7]/C	counterfens_seconds_reg[0]/CE	6.342	1.750	4.592	10.0	sys_clk_pin	sys_clk_pin	
Path 2	3.040	5	6	29	counterclk_cnt_reg[7]/C	counterfens_seconds_reg[1]/CE	6.342	1.750	4.592	10.0	sys_clk_pin	sys_clk_pin	
Path 3	3.040	5	6	29	counterclk_cnt_reg[7]/C	counterfens_seconds_reg[2]/CE	6.342	1.750	4.592	10.0	sys_clk_pin	sys_clk_pin	
Path 4	3.040	5	6	29	counterclk_cnt_reg[7]/C	counterfens_seconds_reg[3]/CE	6.342	1.750	4.592	10.0	sys_clk_pin	sys_clk_pin	
Path 5	3.138	5	6	29	counterclk_cnt_reg[15]/C	counterfens_seconds_reg[0]/CE	6.206	1.648	4.558	10.0	sys_clk_pin	sys_clk_pin	
Path 6	3.138	5	6	29	counterclk_cnt_reg[15]/C	counterfens_seconds_reg[1]/CE	6.206	1.648	4.558	10.0	sys_clk_pin	sys_clk_pin	
Path 7	3.138	5	6	29	counterclk_cnt_reg[15]/C	counterfens_seconds_reg[2]/CE	6.206	1.648	4.558	10.0	sys_clk_pin	sys_clk_pin	
Path 8	3.138	5	6	29	counterclk_cnt_reg[15]/C	counterfens_seconds_reg[3]/CE	6.206	1.648	4.558	10.0	sys_clk_pin	sys_clk_pin	
Path 9	3.285	5	6	29	counterclk_cnt_reg[7]/C	counterclk_cnt_reg[17]/D	6.304	1.749	4.555	10.0	sys_clk_pin	sys_clk_pin	
Path 10	3.665	6	6	29	counterclk_cnt_reg[7]/C	counterfens_seconds_reg[1]/D	6.345	1.890	4.455	10.0	sys_clk_pin	sys_clk_pin	

Figure 15: Setup timing paths (sorted by slack)

To calculate the critical (longest) path delay:

$$longestpath = clock_period - setup_min_slack = 10\text{ ns} - 3.040\text{ ns} = 6.96\text{ ns}$$

From the detailed timing report (Figure 15), the critical path runs from `clk_cnt_reg[7]/C` to `tens_seconds_reg[0]/CE`, with a total delay of 6.342 ns .

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 11	0.234	1	1	1	deb_splitmsb_reg/C	deb_splitoutput_stable_reg/D	0.394	0.329	0.065	0.0	sys_clk_pin	sys_clk_pin	
Path 12	0.236	1	1	1	deb_togglemsb_reg/C	deb_toggleoutput_stable_reg/D	0.395	0.329	0.066	0.0	sys_clk_pin	sys_clk_pin	
Path 13	0.236	1	1	1	deb_trigmsb_reg/C	deb_trigoutput_stable_reg/D	0.395	0.329	0.066	0.0	sys_clk_pin	sys_clk_pin	
Path 14	0.241	1	1	1	deb_resetmsb_reg/C	deb_resetoutput_stable_reg/D	0.365	0.305	0.060	0.0	sys_clk_pin	sys_clk_pin	
Path 15	0.241	1	1	1	deb_samplemsb_reg/C	deb_sampleoutput_stable_reg/D	0.365	0.305	0.060	0.0	sys_clk_pin	sys_clk_pin	
Path 16	0.250	1	1	1	deb_toggleoutput_stable_reg/C	selected_reg/D	0.391	0.285	0.106	0.0	sys_clk_pin	sys_clk_pin	
Path 17	0.283	1	1	4	deb_splitoutput_stable_reg/C	controlstate_reg[0]/D	0.429	0.285	0.144	0.0	sys_clk_pin	sys_clk_pin	
Path 18	0.286	1	1	6	counterclk_cnt_reg[11]/C	counterclk_cnt_reg[12]/D	0.427	0.285	0.142	0.0	sys_clk_pin	sys_clk_pin	
Path 19	0.286	1	1	4	deb_splitoutput_stable_reg/C	controlstate_reg[2]/D	0.432	0.285	0.147	0.0	sys_clk_pin	sys_clk_pin	
Path 20	0.304	0	1	12	counterfens_seconds_reg[1]/C	snapshot_reg[5]/D	0.424	0.223	0.201	0.0	sys_clk_pin	sys_clk_pin	

Figure 16: Hold timing paths (sorted by slack)

The hold timing paths (Figure 16) also show positive slack across all paths, confirming that the design meets hold time requirements as well.

7.3 Project Summary

Question: Provide images of the working project.

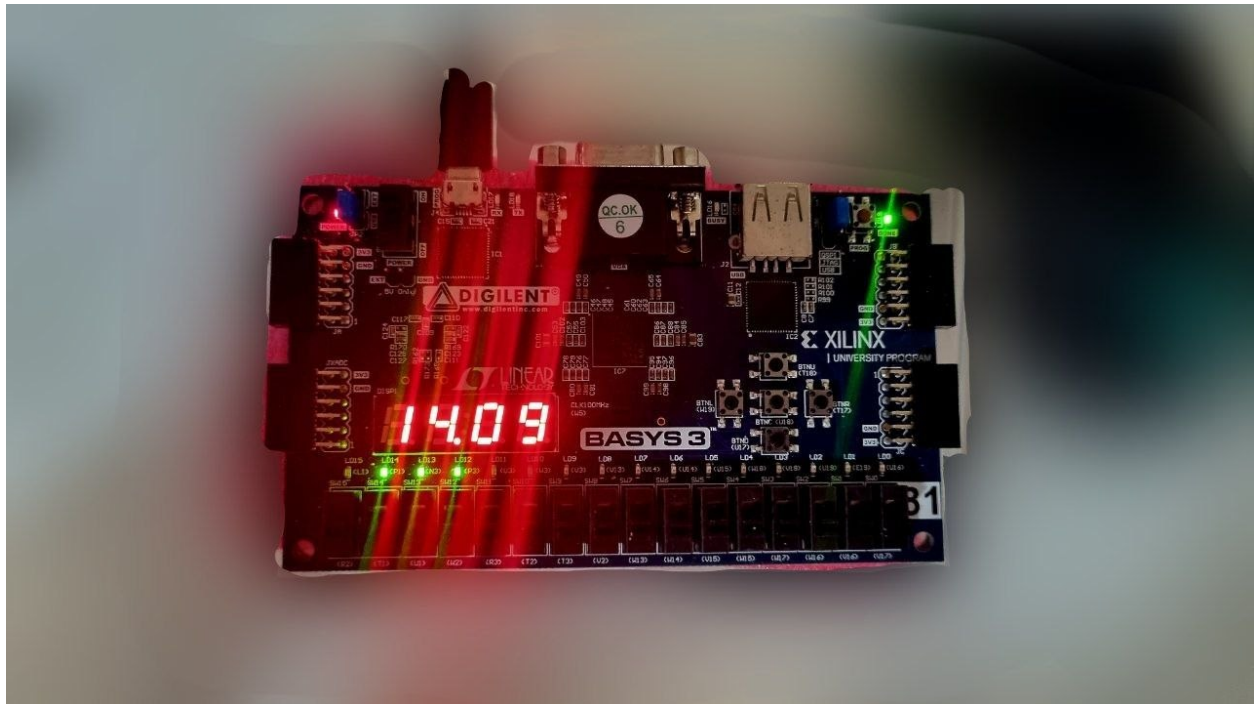


Figure 17: Working Stopwatch with stash on BASYS3 Board